Web Researcher

Project Report

A one-command research automation tool built with Node.js, Firecrawl, and Groq

1. Overview

Web Researcher is a command-line tool that takes a topic and a list of URLs, scrapes each page into clean markdown using Firecrawl, sends the content to Groq's LLaMA 3.3 70B model for analysis, and saves a structured research report to disk. The entire pipeline runs in a single command.

The project was built as Project 02 of a developer tools series — replacing the manual research workflow of opening 15 tabs, skimming articles, and copy-pasting into a doc with a single terminal command that produces a clean, structured output every time.

2. The Problem It Solves

Every project starts with research. The typical manual workflow:

- Search for the topic
- Open 10-15 tabs
- Skim each one
- Copy-paste interesting bits into a doc
- Manually organize and summarize
- Lose track of which source said what

This takes 1-2 hours for a basic competitive analysis. Web Researcher collapses that into one command and under 60 seconds.

3. Architecture

There is no database, no server, no framework. It is a pure Node.js CLI script with two dependencies: groq-sdk and dotenv.

```
Your terminal command
        ↓
  research.js          ← CLI entrypoint, reads topic + URLs from argv
        ↓
  src/scrape.js        ← calls Firecrawl REST API for each URL
        ↓
  Firecrawl API        ← returns clean markdown from each page
        ↓
  src/analyze.js       ← sends all markdown to Groq
        ↓
  Groq LLaMA 3.3 70B   ← analyzes and structures the report
        ↓
  research/[topic].md  ← saved to disk
```

4. File Structure

```
web-researcher/
├── src/
│   ├── scrape.js        ← Firecrawl API wrapper
│   └── analyze.js       ← Groq LLaMA 3 call + prompt
├── research/            ← all generated reports (gitignored)
├── research.js          ← main CLI entrypoint
├── .env                 ← API keys (never committed)
├── .env.example         ← template for new users
├── .gitignore           ← excludes .env, node_modules, research/
├── README.md            ← setup and usage docs
└── package.json         ← type: module, dependencies
```

5. How Each File Works

5.1 research.js — the entrypoint

This is the file you run. It does four things:

1. Loads environment variables from .env via dotenv/config
2. Reads the topic and URLs from process.argv
3. If no URLs are provided, calls Groq to suggest 5 relevant sources automatically
4. Loops through each URL, calls scrapeUrl(), logs progress, then calls analyzePages() and saves the result

The await on scrapeUrl() was a critical fix — the original version was missing it, which meant a Promise object was being stored instead of actual content, causing all pages to appear empty even though the API calls were succeeding.

5.2 src/scrape.js — the Firecrawl wrapper

Calls the Firecrawl REST API at https://api.firecrawl.dev/v1/scrape with a POST request. Sends the URL and requests markdown format. Returns { url, content, error } for every page so failures are handled gracefully without crashing the whole run.

The original plan was to use Firecrawl as a CLI tool via npm install -g firecrawl, but the npm package does not ship a CLI binary. The fix was to call the REST API directly using Node's built-in fetch.

5.3 src/analyze.js — the Groq call

Initializes the Groq client and sends all scraped content to llama-3.3-70b-versatile with a structured system prompt. Each page's content is capped at 4,000 characters before being sent, keeping total context reasonable for multi-URL runs. The model is prompted to produce a report with five fixed sections and to flag conflicts between sources with a warning marker.

6. Tech Stack

Layer	Tool	Why
Runtime	Node.js v26	Native fetch, ES modules, no extra deps
LLM	Groq - LLaMA 3.3 70B	Fast inference, free tier, 128k context
Scraping	Firecrawl REST API	Returns clean markdown, handles JS-heavy sites
Config	dotenv	Standard .env loading
Output	fs.writeFileSync	Plain markdown files, no database needed

7. Bugs Fixed During Development

Bug 1 — Missing await on scrapeUrl()

The original research.js called scrapeUrl(url) without await. Because scrapeUrl is an async function, this stored a pending Promise in the pages array instead of the resolved result. The filter pages.filter(p => p.content) then saw empty content on every page and reported 0/N pages scraped, even though the Firecrawl API was returning good data.

Fix: added await before scrapeUrl(url).

Bug 2 — Firecrawl has no CLI binary

The project brief recommended npm install -g firecrawl and calling it via execSync. The package exists on npm but ships no executable. Running firecrawl --version returned command not found even after a successful install.

Fix: removed the CLI wrapper entirely and called the Firecrawl REST API directly using fetch.

8. Key Decisions

Why Groq instead of OpenAI or Anthropic?

Groq's free tier is generous and has no credit card requirement. LLaMA 3.3 70B on Groq is fast enough that the analysis step takes under 10 seconds even for large scrapes.

Why Firecrawl instead of plain fetch + cheerio?

Many modern sites are JavaScript-rendered and return empty HTML to a plain fetch request. Firecrawl handles this transparently and returns clean markdown without requiring a headless browser setup. The free tier gives 500 scrapes per month.

Why markdown output instead of JSON or HTML?

Markdown is the most portable format — it renders in GitHub, Notion, Obsidian, VS Code, and any text editor. Reports saved as markdown can be committed to a repo, pasted into a doc, or fed into another LLM call downstream.

Why CLI instead of a web UI?

Speed and composability. A CLI tool can be piped into other tools, scheduled with cron, or called from a shell script. A web UI would add complexity without adding value for the target user.

Why ES modules (type: module)?

ES module syntax (import/export) is cleaner and is the direction Node.js is heading. The tradeoff is that require() doesn't work, but all packages used here support ESM.

9. Output Format

Every report follows the same five-section structure regardless of topic:

```
# [Topic] -- Research Report

## Key facts & data points
## Different perspectives & approaches
## Conflicts & gaps
## Sources
## 3 questions worth investigating further
```

This consistency is enforced by the system prompt in analyze.js. Having a fixed structure means reports are predictable, easy to skim, and straightforward to parse programmatically.

10. Limitations

Limitation	Detail	Workaround
Firecrawl free tier	500 scrapes/month. At 5 URLs per run = ~100 runs/month	Upgrade to paid plan for heavier use
LLM hallucinated URLs	Groq occasionally suggests URLs that don't exist when no URLs are provided	Always pass real URLs; treat auto-suggested ones as a starting point
4,000 char cap per page	Content truncated before sending to Groq. May miss content on dense pages	Raise the slice limit in analyze.js - LLaMA supports 128k tokens
No deduplication	Overlapping content across sources can cause repeated analysis in the report	Pass distinct, non-overlapping URLs

11. How to Extend This

Add search before scraping

Integrate a search API (Tavily, Brave Search, or SerpAPI) so you can run node research.js "topic" without providing URLs and get results from a real search query rather than LLM-suggested URLs.

Schedule weekly runs

Add a cron job to run the researcher on a fixed topic every week and commit the report to a git repo automatically. Good for tracking how a market or technology evolves over time.

Chain into a product brief

After the report is saved, make a second Groq call that reads the report and generates a product brief, competitive positioning doc, or executive summary.

HTML output

Add a --format html flag that converts the markdown report into a styled HTML file using a template, making it shareable without a markdown renderer.

Slack or email delivery

Add a flag to post the report summary to a Slack channel or send it as an email after each run, so research can be triggered and delivered without opening a terminal.

12. Usage Reference

```
# Research with specific URLs
node research.js "topic" url1 url2 url3

# Research with Groq-suggested URLs
node research.js "topic"

# View a saved report
cat research/[topic].md

# Example
node research.js "AI coding tools 2026" \
  https://cursor.com \
  https://windsurf.com \
  https://github.com/features/copilot
```

13. Setup Reference

```
git clone https://github.com/yourusername/web-researcher.git
```

```
cd web-researcher
npm install
cp .env.example .env
# add GROQ_API_KEY and FIRECRAWL_API_KEY to .env
node research.js "your topic" https://example.com
```

API Keys:

- Groq: console.groq.com -> API Keys -> Create (free, no credit card)
- Firecrawl: firecrawl.dev -> Dashboard -> API Keys (free tier: 500 scrapes/month)